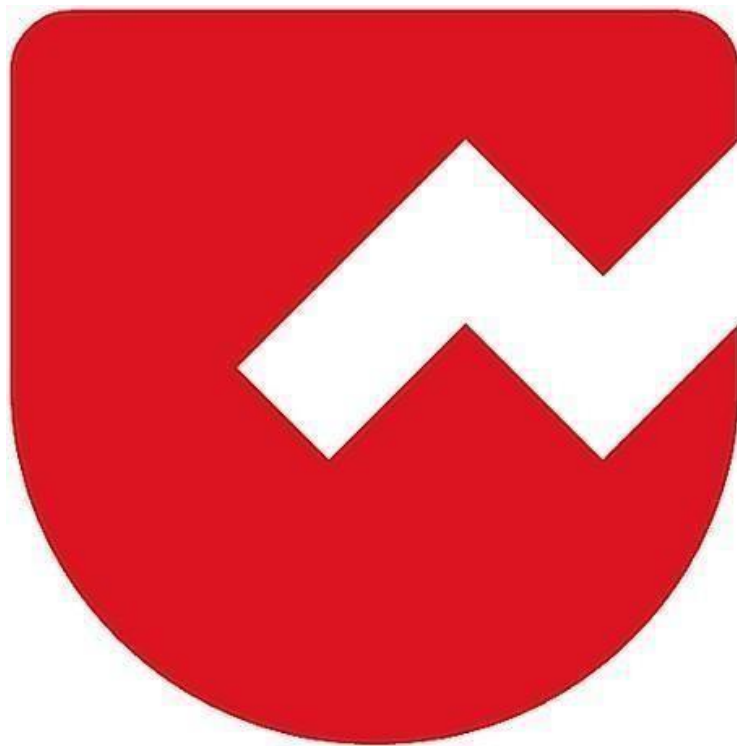


Zoho Schools for Graduate Studies

Notes



Synchronization in Multithreading:

- ⑩ Synchronization means controlling access to shared resources so that only one thread can use it at a time.
- ⑩ It prevents data inconsistency and race conditions.

Synchronization using join() :

- ⑩ join() is **not locking a resource** like synchronized.
- ⑩ join() synchronizes threads based on execution order.
- ⑩ It makes **one thread wait until another thread finishes**.
- ⑩ synchronized → **controls shared data**.
- ⑩ join() → **controls thread sequence**.

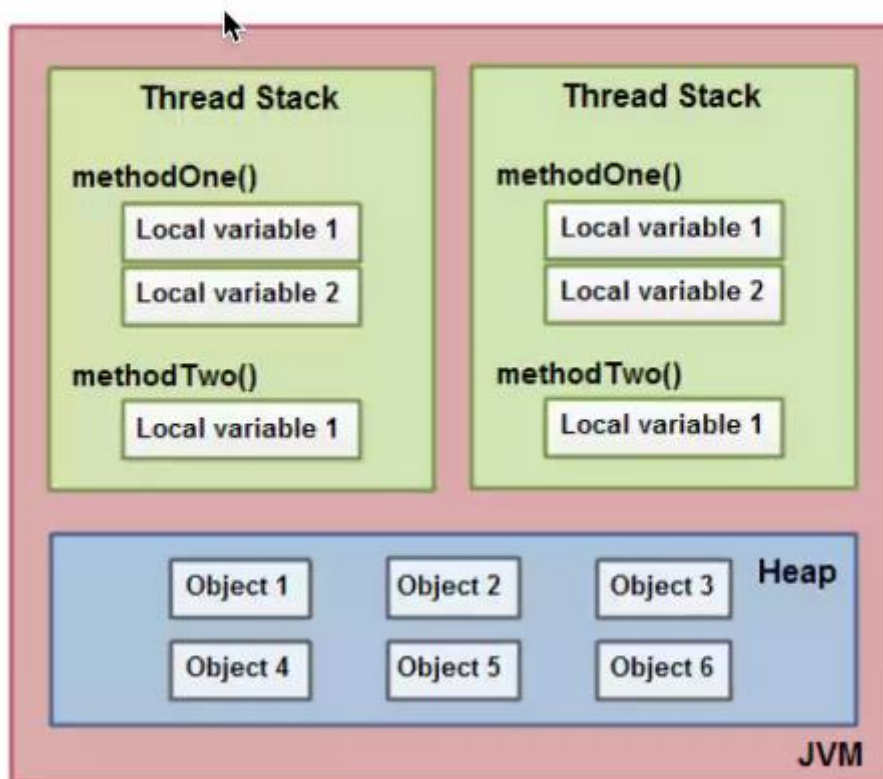
**<threadInstance>.join blocks the calling thread until the
<threadInstance> has finished**

Join supports .interrupts too

Java Memory Model (JMM):

- ⑩ How memory is structured
- ⑩ How threads access memory
- ⑩ When changes made by one thread are visible to other threads

Java Memory Model



- ⑩ Order of execution can be rearranged by compiler for optimisation .This will cause issue when shared variables are used inside threads.

Why did the problem occur?

Story behind shorthand increment/decrement //CPU instructions.

Read a, r1

Add r1, 1

Store r1, a

Synchronization by Mutual Exclusion:

Synchronized methods

Synchronized blocks //multiple lock not allowed

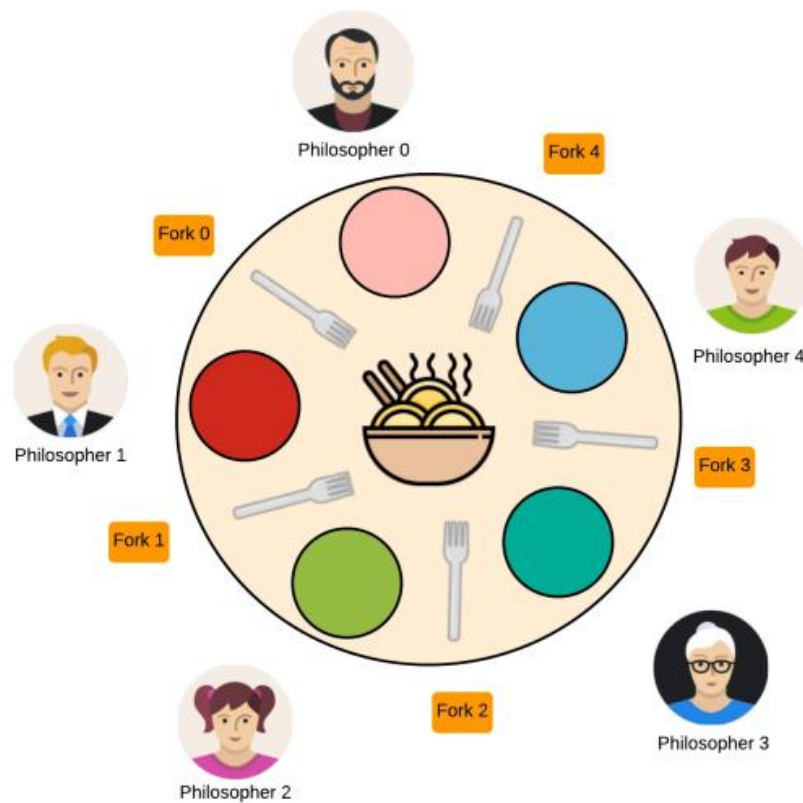
Volatile:

It ensures value of the variable is always read from and written to the main memory,

rather than from the CPU cache of individual threads

Note: Volatile cannot be used for methods and classes

Dining Philosopher's Problem:



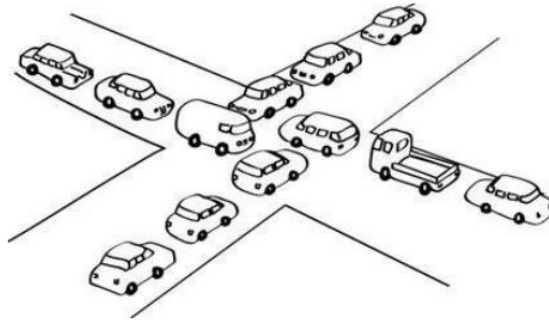
Deadlock:

Note: When synchronization goes wrong

Deadlock is a situation in multithreading where two or more threads are permanently blocked, because each thread is waiting for a resource held by another thread.

Pic 1:

pic 2:



class



```
DeadlockDemo {  
    static Object lock1 = new Object();  
    static Object lock2 = new Object();  
  
    public static void main(String[] args) {  
  
        Thread t1 = new Thread() -> {  
            synchronized (lock1) {  
                System.out.println("Thread 1 locked lock1");  
            }  
        }  
    }  
}
```

```

        try { Thread.sleep(100); } catch (Exception e) {}

        synchronized (lock2) {
            System.out.println("Thread 1 locked lock2");
        }
    }
});

```

```

Thread t2 = new Thread() -> {
    synchronized (lock2) {
        System.out.println("Thread 2 locked lock2");
        try { Thread.sleep(100); } catch (Exception e) {}

        synchronized (lock1) {
            System.out.println("Thread 2 locked lock1");
        }
    }
});

```

```

t1.start();
t2.start();
}
}

```

Other things to explore:

- ⑩ ReentrantLock

- ⑩ java.util.concurrent package
- ⑩ Collections.<synchronized methods>